

ChatGPT



what kind of ML projects company look for during interviews, like is time based data more important?

Companies usually care less about *the exact domain* of your ML project and more about whether the project demonstrates **real ML engineering and problem-solving ability**.

Time-series projects (stock prediction, forecasting, sensor data, etc.) are valuable, but they are **not automatically more important** than other ML projects. What matters is:

- complexity,
- business relevance,
- data handling,
- evaluation rigor,
- deployment understanding,
- and how deeply *you* understand the project.

What Companies Actually Look For

1. End-to-End Thinking (Most Important)

A strong project shows the full ML lifecycle:

Problem → Data → Cleaning → Feature Engineering → Modeling → Evaluation → Deployment/Monitoring

Many candidates only train a model in a notebook.

Interviewers prefer candidates who can explain:

- Why this problem matters
- Why the dataset was difficult
- What failed
- Why you chose certain models
- How you validated results
- How the model would work in production

Types of Projects That Impress Recruiters

A. Tabular ML Projects (Very Valuable)

These are actually the most common in industry.

Examples:

- fraud detection
- churn prediction
- loan default prediction
- customer segmentation
- ad click prediction
- sales prediction

Skills demonstrated:

- feature engineering
- handling missing values
- imbalanced datasets
- XGBoost/LightGBM
- business metrics

These projects are highly respected because real companies mostly work with structured data.

B. Time-Series Projects (Good if Done Properly)

Examples:

- demand forecasting
- traffic prediction
- energy consumption
- anomaly detection
- stock volatility
- server monitoring

Time-series is valuable because it shows:

- temporal reasoning
- leakage prevention
- forecasting validation
- sequential modeling

But many student projects are weak because they:

- use random train/test split (wrong for time-series)
- predict stock prices unrealistically
- ignore drift and seasonality

A strong time-series project includes:

- rolling validation
- seasonality analysis
- drift handling
- forecasting horizons
- proper metrics like MAE/MAPE/RMSE

Good models:

- ARIMA
 - Prophet
 - XGBoost with lag features
 - LSTM/Transformers (only if justified)
-

C. NLP Projects (Very Popular)

Examples:

- RAG chatbot
- document search
- resume screening
- sentiment analysis
- summarization
- multilingual systems

Especially valuable now because companies want:

- LLM integration
- embeddings/vector DBs
- prompt engineering
- retrieval pipelines

Strong NLP projects often outperform generic CNN image projects in interviews today.

D. Recommendation Systems (Extremely Valuable)

Examples:

- movie recommendation
- ecommerce recommendation
- music recommendation

Why companies like them:

- combines ML + system design
- real business impact
- ranking problems
- personalization

Shows maturity beyond basic classification.

E. MLOps / Deployment Projects (Huge Advantage)

This is where candidates differentiate themselves.

If your project includes:

- FastAPI/Flask serving
- Docker
- CI/CD
- MLflow
- monitoring
- AWS/GCP
- pipelines

you immediately stand out.

Many candidates know modeling.

Few know production ML.

What Makes a Project “Interview-Worthy”

A good project should have at least 4–5 of these:

Signal	Why It Matters
Real-world messy data	Shows practical skill
Non-trivial preprocessing	Demonstrates understanding
Baseline comparison	Shows scientific thinking
Proper evaluation	Avoids “accuracy-only” trap
Error analysis	Shows maturity

Signal	Why It Matters
Feature engineering	Very important
Deployment/API	Industry relevance
Scalability discussion	Strong engineering signal
Business metric alignment	Product thinking
Clear GitHub/readme	Communication skill

What Interviewers Often Ask

Expect questions like:

- Why did you choose this model?
- What alternatives did you try?
- How did you avoid overfitting?
- What metric mattered most?
- What would break in production?
- How would you scale this?
- How would retraining work?
- What are the failure modes?

If you cannot answer these deeply, even a fancy project loses value.

Best Projects for Different Roles

For Data Scientist Roles

Focus on:

- experimentation
- feature engineering
- statistics
- business insights

Best projects:

- churn prediction
- forecasting
- recommendation systems

For ML Engineer Roles

Focus on:

- deployment
- pipelines
- APIs
- scalability

Best projects:

- real-time inference systems
 - RAG pipelines
 - ML serving architecture
 - MLOps
-

For AI/LLM Roles

Focus on:

- RAG
- fine-tuning
- embeddings
- agent systems

Very hot currently.

What Is Overrated

These projects are extremely common now:

- MNIST digit classifier
- simple cat-vs-dog CNN
- Titanic dataset
- Iris dataset
- basic stock prediction notebook
- copy-paste ChatGPT wrapper

They are okay for learning, but weak for interviews unless extended substantially.

What I'd Recommend in 2026

A very strong portfolio today is:

1. One structured/tabular ML project
2. One LLM/NLP project
3. One deployed production-style project

Optional:

- time-series OR recommendation system

That combination signals breadth + engineering depth.

Example of a Strong Interview Project

Demand Forecasting System

Problem:

Predict retail demand for next 14 days.

Includes:

- time-series forecasting
- feature engineering
- holiday effects
- drift analysis
- XGBoost + Prophet comparison
- FastAPI deployment
- Docker
- dashboard
- retraining pipeline

This is significantly stronger than:

“I trained an LSTM on stock prices.”

Bottom Line

Time-series projects are respected, but they are **not inherently superior**.

Companies care more about:

- practical relevance,
- engineering depth,
- evaluation rigor,

- and whether you can discuss tradeoffs intelligently.

A well-executed tabular ML or NLP project often beats a flashy deep learning project with weak methodology.



So I should not worry about the topic but what goes inside it

Exactly.

The **execution quality** matters far more than the topic itself.

A mediocre “hot” topic project is weaker than a well-engineered “boring” project.

What Recruiters Actually Infer From a Project

They are subconsciously evaluating:

Question	What They Want
Can this person handle messy data?	Practical skill
Can they think scientifically?	Experimentation mindset
Can they debug failures?	Engineering maturity
Can they explain tradeoffs?	Communication
Can they productionize ML?	Industry readiness
Did they copy this from YouTube?	Authenticity check

The topic is just the vehicle.

Example

Weak “Cool” Project

“I built a stock prediction LSTM.”

But:

- random train/test split
- no leakage prevention
- no baseline
- no deployment
- copied architecture
- no explanation of failures

This often performs poorly in interviews.

Strong “Ordinary” Project

“I built a customer churn prediction system.”

But:

- extensive feature engineering
- imbalance handling
- SHAP explainability
- ablation experiments
- MLflow tracking
- FastAPI deployment
- monitoring discussion
- business metric optimization

This candidate usually looks much stronger.

What Separates Strong Candidates

Strong candidates discuss:

- *why* they made decisions
- what failed
- tradeoffs
- production concerns
- limitations
- future improvements

Weak candidates only discuss:

- architecture names
 - accuracy numbers
 - libraries used
-

A Good Rule

Your project should answer:

Why does this exist?
Why is this hard?

Why is this approach reasonable?
How do we know it works?
How would it survive in production?

If you can answer those well, the topic becomes secondary.

What Makes Interviewers Lean Forward

Usually one of these:

1. Real Constraints

Examples:

- low latency
 - small dataset
 - class imbalance
 - noisy labels
 - memory limits
-

2. Thoughtful Evaluation

Examples:

- cross-validation strategy
 - temporal validation
 - precision/recall tradeoffs
 - calibration
 - error analysis
-

3. Engineering Depth

Examples:

- APIs
 - Docker
 - pipelines
 - monitoring
 - retraining systems
-

4. Business Understanding

Example:

“False negatives cost more than false positives, so I optimized recall.”

That sounds industry-ready.

One More Important Thing

Interviewers can detect copied projects *very quickly*.

Especially:

- generic Kaggle notebooks
- standard CNN tutorials
- cloned GitHub repos
- ChatGPT-generated pipelines with no understanding

So choose projects where you genuinely explored the problem.

Depth > trendiness.

The Best Strategy

Pick projects that are:

- interesting enough that you stay motivated
- complex enough to discuss deeply
- practical enough to resemble real ML work

Then make them:

- rigorous,
- reproducible,
- explainable,
- and deployable.

That combination is what gets interviews.



See I am thinking of making 1 churn based project, 1 recommender project and one project where target is numerical. Now I have to start with regression one, but how do I get messy data for any topic

That's a very good project mix.

- **Churn project** → classification + business ML
- **Recommender system** → ranking/personalization
- **Regression project** → forecasting/value prediction

That already looks more industry-oriented than most beginner portfolios.

Now for your actual concern:

“How do I get messy real-world data?”

This is the correct question to ask.

Because clean Kaggle datasets are rarely what companies deal with.

The Secret: You Usually CREATE the Messiness

Real ML skill is often demonstrated by:

- identifying data quality issues,
- engineering features,
- fixing inconsistencies,
- and designing pipelines.

Even if the original dataset is decent, you can make the project realistic.

Best Sources for Realistic Data

1. Kaggle (Still Fine)

But avoid:

- tutorial datasets
- “cleaned” versions
- beginner competitions

Instead search for:

- retail
- ecommerce
- transactions
- telecom
- logistics

- healthcare
- sensor data

Good datasets often already contain:

- missing values
- duplicates
- skewed distributions
- inconsistent timestamps
- outliers

2. Government/Open Data Portals (Underrated)

These are often genuinely messy.

Examples:

- traffic
- pollution
- energy usage
- weather
- public transport
- agriculture

Sources:

- data.gov.in
- [NYC Open Data](https://data.cityofnewyork.org/)
- [UCI ML Repository](https://archive.ics.uci.edu/)

These datasets frequently have:

- corrupted entries
- inconsistent formatting
- sparse records
- temporal gaps

Which is actually good for interviews.

3. APIs + Web Scraping (Very Strong Signal)

This is closer to real industry workflows.

Examples:

- weather APIs
- crypto APIs
- ecommerce scraping
- sports stats
- movie ratings
- public finance APIs

Now your project naturally includes:

- ingestion pipelines
- schema inconsistencies
- missing fields
- update scheduling

That becomes impressive.

For Your Regression Project — Strong Ideas

Here are some high-value options.

Option 1 — Demand Forecasting (Highly Recommended)

Predict:

- sales
- inventory demand
- order volume

This is extremely industry relevant.

Dataset sources:

- Walmart sales
- Rossmann store sales
- Favorita grocery sales

You can intentionally include:

- holidays
- promotions
- missing dates
- store-level variability
- drift

This project can become very strong.

Option 2 — House Price Prediction (If Done Properly)

Normally overdone.

BUT it becomes strong if you:

- combine multiple datasets
- geospatial features
- temporal trends
- outlier handling
- explainability

Most people stop at linear regression.

You go further.

Option 3 — Ride Fare / Delivery Time Prediction

Very industry-like.

Predict:

- taxi fare
- ETA
- delivery duration

Messiness naturally exists:

- traffic anomalies
- GPS noise
- missing routes
- weather effects

Very realistic ML problem.

Option 4 — Energy Consumption Forecasting

Excellent time-series regression.

Predict:

- electricity usage
- server load
- power demand

Shows:

- seasonality
- drift
- lag features
- forecasting validation

Strong engineering potential.

How To “Add Realism” to Any Dataset

This is what good candidates do.

Add Missingness

Simulate:

- unavailable sensors
- failed logging
- delayed records

Then discuss:

- imputation strategy
 - robustness
-

Add Temporal Drift

Example:

- consumer behavior changes over time
- inflation
- seasonal demand

Now you can discuss:

- retraining
- monitoring
- drift detection

Very valuable.

Merge Multiple Data Sources

Example:

```
sales + holidays + weather + promotions
```

This immediately upgrades project complexity.

Create Feature Pipelines

Instead of:

```
Python
df["x"] = ...
```

Use:

- sklearn pipelines
- feature stores
- reusable preprocessing

Interviewers like this.

What Makes a Regression Project Strong

Most candidates only do:

```
train model → report RMSE
```

A strong project includes:

Component	Strong Signal
Baseline models	Scientific rigor
Feature engineering	Practical skill
Error analysis	Maturity
Drift discussion	Production awareness
Explainability	Business readiness
Deployment	Engineering
Monitoring ideas	Senior thinking

My Recommendation For You

Given your planned portfolio:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Best Combination

1. Churn Prediction

Use:

- XGBoost/LightGBM
 - SHAP
 - imbalance handling
-

2. Recommendation System

Use:

- collaborative filtering
 - hybrid recommendation
 - ranking metrics
-

3. Demand Forecasting / Time-Series Regression

This fits perfectly.

Because together you show:

- classification
- ranking
- regression/forecasting

That's excellent breadth.

One Important Tip

Don't optimize for:

"Which project sounds coolest?"

Optimize for:

"Which project lets me discuss the deepest engineering and ML tradeoffs?"

That's what interviewers remember.



how about appliances price predictor, I have found a website from where I can scrape the data since it has infinite scrolling on the same page

Yes — that can actually become a **very strong ML project** if you design it properly.

And the fact that you're scraping the data yourself is already a big positive signal.

That immediately makes the project more realistic than:

"Downloaded CSV from Kaggle."

Why Appliance Price Prediction Is Good

It naturally includes:

Component	Why It's Valuable
Web scraping	Data engineering
Messy product specs	Real-world preprocessing
Regression	Core ML
Feature extraction	NLP/tabular hybrid
Outliers	Practical modeling
Deployment potential	Product thinking

This can absolutely become interview-worthy.

What Would Make It Strong

1. Don't Make It "Just Regression"

The real value is:

scraping → cleaning → feature extraction →
modeling → explainability → deployment

That full pipeline is impressive.

2. The Dataset Will Naturally Be Messy

This is GOOD.

You'll likely encounter:

- inconsistent naming
- missing specs
- duplicate products
- mixed units
- discount pricing

- weird categorical values

Example:

```
"1.5 Ton AC"
"1.5T Split AC"
"Split AC 1.5 Ton"
```

Now you must standardize.
That's real ML work.

3. Feature Engineering Opportunities Are Excellent

Suppose you scrape:

- brand
- rating
- reviews
- energy rating
- capacity
- inverter/non-inverter
- smart features
- warranty
- product description

Now you can:

- extract structured features from text
- engineer premium-brand signals
- compute review sentiment
- derive price-per-capacity metrics

This becomes much richer than basic regression.

This Is What Would Impress Interviewers

Example Architecture

```
Scraper (Selenium/Playwright)
  ↓
Raw Product Data
  ↓
Cleaning + Parsing Pipeline
  ↓
Feature Engineering
  ↓
```

```
Regression Models
↓
Model Explainability
↓
FastAPI Prediction Service
↓
Dashboard/Web App
```

That looks industry-level.

VERY Important: Don't Predict Current Listed Price Only

That's too shallow.

Instead frame it like:

Better Problem Statements

Option A — Fair Price Estimation

Predict the "expected fair market price" of appliances from specifications.

Good because:

- enables anomaly detection
- business relevance

Option B — Discount Prediction

Predict whether an appliance is overpriced or underpriced.

Very practical.

Option C — Resale Price Prediction

Even stronger if you can get second-hand data.

This introduces:

- depreciation
- condition
- temporal effects

Very realistic ML.

Infinite Scroll Scraping Is Actually Valuable

Because now you deal with:

- dynamic loading
- async content
- anti-bot handling
- pagination logic

That's useful engineering experience.

Tools:

- Selenium
- Playwright (better modern choice)
- BeautifulSoup after render
- Scrapy if scalable

Biggest Opportunity: Text Features

This project becomes MUCH stronger if you use product titles/descriptions intelligently.

Example:

```
"LG 1.5 Ton 5 Star Dual Inverter WiFi Split AC"
```

You can extract:

- brand = LG
- tonnage = 1.5
- energy_rating = 5
- inverter = yes
- smart = yes

That's excellent preprocessing work.

Models You Could Compare

Start simple.

Baselines

- Linear Regression
- Ridge/Lasso

Strong Models

- Random Forest
- XGBoost
- LightGBM
- CatBoost

Tabular boosting models usually dominate here.

What Will Differentiate You

Add Explainability

Use:

- SHAP
- feature importance
- partial dependence plots

Then discuss:

“Energy rating and inverter technology had the largest positive price impact.”

Interviewers like this.

Potential Advanced Additions

1. Time-Aware Pricing

Track prices over time.

Now you introduce:

- price volatility
- seasonal discounts
- drift

Very strong addition.

2. Price Recommendation Engine

Suggest:

“This product is overpriced by 12%.”

Now the project becomes product-oriented.

3. Retrieval + Similar Products

Use embeddings or TF-IDF to find comparable appliances.

Now your recommender project and regression project connect.

One Warning

Avoid:

“Accuracy = 98%”

Regression doesn’t work like that.

Focus on:

- MAE
 - RMSE
 - percentage error
 - calibration
 - robustness
-

Overall Verdict

This is a genuinely good idea because it combines:

- scraping
- messy real-world data
- feature engineering
- regression
- deployment potential

If executed well, it is much stronger than many standard ML portfolio projects.